11 机器代码布局优化

CPU 前端（FE）负责获取和解码指令，并将它们传递给乱序执行的后端。随着新型处理器获得更多的执行“马力(horsepower)”，CPU FE 需要足够强大以保持机器平衡。如果 FE 无法跟上指令供应，后端将被低效利用，整体性能将受到影响。这就是为什么 FE 设计为始终领先于实际执行，以平滑可能出现的任何问题 (hiccup)，并始终准备好待执行的指令。例如，2016 年发布的英特尔 Skylake 可以在每个周期内获取多达 16 条指令。

大多数时候，CPU FE 的效率低下可以描述为后端等待执行指令，但 FE 无法提供它们的情况。结果，CPU 周期在没有进行任何实际有用工作的情况下被浪费。回想一下，现代 CPU 每个周期可以处理多条指令，现在范围从 4- 到 8-wide。不是所有可用插槽都被填满的情况经常发生。这代表了数据库、编译器、网络浏览器等多个领域应用程序的效率低下来源。

TMA 方法论通过 前端瓶颈(`Front-End Bound`) 指标捕捉 FE 性能问题。它代表了 CPU FE 无法向 BE 提供指令的周期百分比，而它本可以接受它们。大多数实际应用程序都会经历非零的`Front-End Bound`指标，这意味着一定百分比的运行时间将因次优的指令获取和解码而损失。低于 10% 是常态。如果你看到“Front-End Bound”指标超过 20%，那么绝对值得花时间解决它。

FE 无法向执行单元提供指令的原因可能有很多。大多数时候，这是由于代码布局不佳，导致 I-cache 和 ITLB 利用不佳。拥有大型代码库的应用程序，例如数百万行代码，特别容易受到 FE 性能问题的影响。在本章中，我们将探讨一些典型的优化措施，以改善机器代码布局并提高程序的整体性能。

11.1 机器代码布局

当编译器将源代码转换为机器代码时，它会生成一个线性的字节序列。Listing 11.1 显示了一个小段 C++ 代码的二进制布局示例。一旦编译器完成生成汇编指令，它需要对它们进行编码并按顺序排列在内存中。

代码清单: 机器代码布局示例

```
  C++ Code        Assembly Listing      Disassembled Machine Code
  ........        ................      ........................
if (a <= b)        ; a is in edi        401125 cmp esi, edi
  bar();           ; b is in esi        401128 jb 401131
else               cmp esi, edi         40112a call bar
  baz();           jb .label1           40112f jmp 401136
                   call bar()           401131 call baz
                   jmp .label2          401136 ...
                 .label1:
                   call baz()
                 .label2:
                   ...
```

代码在二进制文件中放置的方式称为 机器代码布局。请注意，对于同一个程序，可以以许多不同的方式布局代码。对于 Listing 11.1 中的代码，编译器可能决定反转分支，以便首先调用`baz`。此外，函数`bar`和`baz`的主体可以以两种不同的顺序放置：我们可以先在二进制文件中放置`bar`，然后放置`baz`，或者反转顺序。这会影响指令在内存中放置的偏移量，进而可能会影响生成的二进制文件的性能，正如您将在后面看到的。在本章的以下部分中，我们将介绍一些机器代码布局的典型优化。

11.2 基本块

基本块是一系列具有单个入口和单个出口的指令。图 67 显示了一个基本块的简单示例，其中 MOV 指令是一个入口，而 JA 是一个退出指令。虽然一个基本块可以有一个或多个前驱和后继，但中间的指令不能进入或退出基本块。

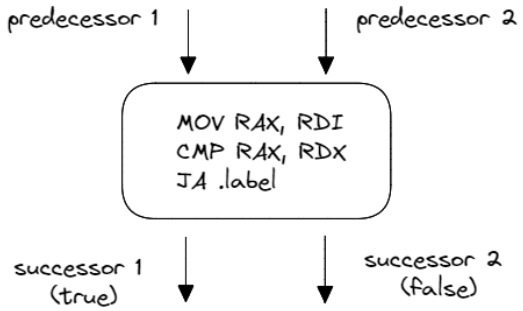


Figure 67: 汇编指令的基本块。

保证基本块中的每条指令都会被执行一次。这是一个重要的属性，被许多编译器转换所利用。例如，它极大地减少了控制流图分析和转换的问题，因为对于某些问题类别，我们可以将基本块中的所有指令视为一个实体。

11.3 基本块布局

假设我们在程序中有一个热路径，其中有一些错误处理代码（coldFunc）：

```
// 热路径
if (cond)
  coldFunc();
// 再次热路径
```

图 68 显示了这段代码的两种可能的物理布局。如果没有提供任何提示，图 68a 是大多数编译器默认会生成的布局。如果我们反转条件 cond 并将热代码放置为 fall through，则可以实现图 68b 中所示的布局。

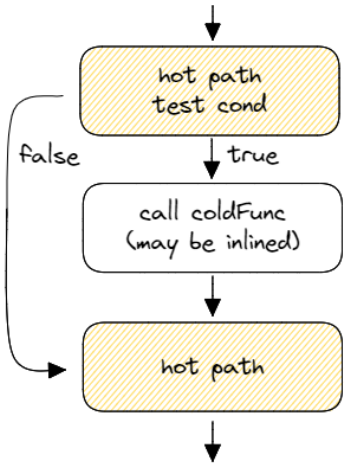
哪种布局更好？这取决于 cond 通常是真还是假。如果 cond 通常为真，那么我们最好选择默认布局，因为否则我们会执行两次跳转而不是一次。另外，在一般情况下，如果 coldFunc 是一个相对较小的函数，我们希望它被内联。但是，在这个特定的例子中，我们知道 coldFunc 是一个错误处理函数，可能不会经常执行。通过选择布局 68b ，我们在代码的热部分之间保持 fall through，并将 taken branch 转换为 not taken branch。

图 68b 中呈现的布局性能更好，原因有几个。首先，图 68b 中的布局更好地利用了指令和 μop-cache（DSB，参见 Section 3.8.1）。所有热代码都连续，没有缓存行碎片：L1I-cache 中的所有缓存行都被热代码使用。μop-cache 也是如此，因为它也是基于底层代码布局进行缓存的。其次，taken branch 对于 fetch 单元来说也更昂贵。CPU 前端会连续获取字节块，因此每次 taken jump 都意味着 jump 之后的字节是无用的。这会降低最大有效提取吞吐量。最后，在某些架构上，not taken branch 比 taken branch 便宜。例如，Intel Skylake CPU 每周期可以执行两个 untaken branch，但每两周期只能执行一个 taken branch。[182]

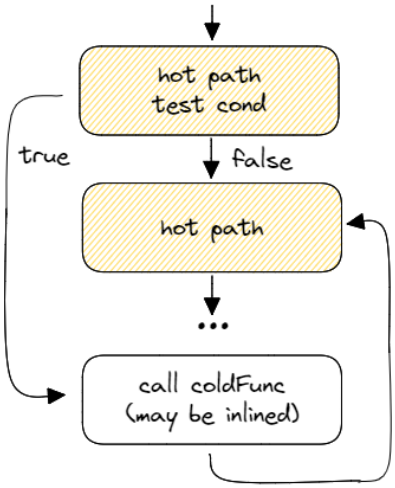
为了建议编译器生成改进版本的机器代码布局，可以使用 [[likely]][183] 和 [[unlikely]] 属性提供提示，该属性从 C++20 开始可用。使用此提示的代码如下所示：

```
// 热路径
if (cond) [[unlikely]]
```

[182] 不过，有一个特殊的微型循环优化，可以使非常小的循环每个周期执行一个 taken branch。
[183] C++ 标准 [[likely]] 属性：https://en.cppreference.com/w/cpp/language/attributes/likely。



(a) 默认布局



(b) 改进的布局

Figure 68: 上面代码片段的两种机器代码布局版本。

```
  coldFunc();
// 再次热路径
```

在上面代码中，[[unlikely]] 提示将指示编译器 cond 不太可能为真，因此编译器应相应地调整代码布局。在 C++20 之前，开发人员可以使用 __builtin_expect: https://llvm.org/docs/BranchWeightMetadata.html#builtin-expect[184] 构造，他们通常自己创建 LIKELY wrapper 提示以使代码更具可读性。例如：

```
#define LIKELY(EXPR) __builtin_expect((bool)(EXPR), true)
#define UNLIKELY(EXPR) __builtin_expect((bool)(EXPR), false)
// 热路径
if (UNLIKELY(cond)) // NOT
  coldFunc();
// 再次热路径
```

优化编译器不仅会在遇到“likely/unlikely”提示时改进代码布局。他们还会在其他地方利用这些信息。例如，当应用 [[unlikely]] 属性时，编译器将阻止内联 coldFunc，因为它现在知道它不太可能经常被执行，并且优化它的大小更有利，即只留下一个 CALL 到这个函数。插入 [[likely]] 属性对于 switch 语句也是可能的，如 Listing 184 所示。

代码清单:switch 语句中可能使用的属性

```
for (;;) {
  switch (instruction) {
              case NOP: handleNOP(); break;
    [[likely]] case ADD: handleADD(); break;
              case RET: handleRET(); break;
    // handle other instructions
  }
}
```

使用此提示，编译器将能够稍微重新排序代码并优化热交换以更快地处理 ADD 指令。

11.4 基本块对齐

有时，性能会根据指令在内存中的偏移量而发生显着变化。考虑 Listing 11.4 中提供的简单函数以及使用 -O3 -march=core-avx2 -fno-unroll-loops 编译时对应的机器码。为了说明这个想法，循环展开被禁用了。

代码清单: 基本的块对齐

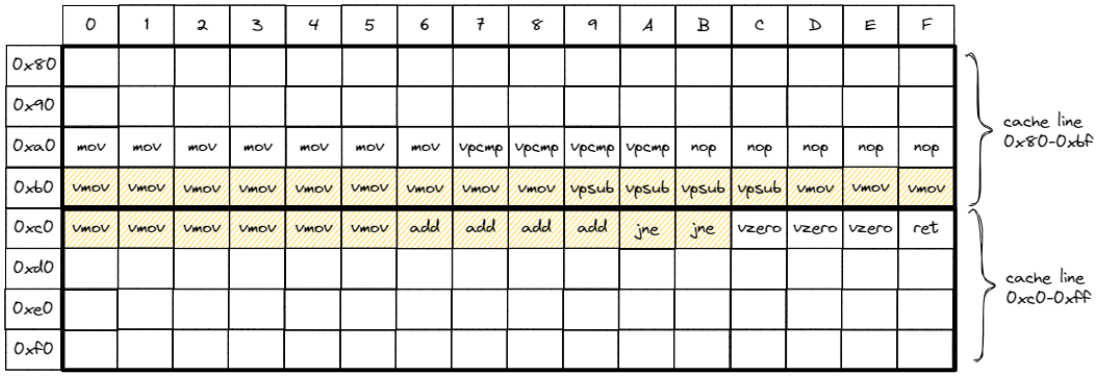
```
void benchmark_func(int* a) {
  for (int i = 0; i < 32; ++i)
    a[i] += 1;
}
```

```
00000000004046a0 <_Z14benchmark_funcPi>:
4046a0: mov rax,0xfffffffffffffff80
4046a7: vpcmpeqd ymm0,ymm0,ymm0
4046ab: nop DWORD [rax+rax+0x0]
4046b0: vmovdqu ymm1,YMMWORD [rdi+rax+0x80] # loop begins
4046b9: vpsubd ymm1,ymm1,ymm0
4046bd: vmovdqu YMMWORD [rdi+rax+0x80],ymm1
4046c6: add rax,0x20
4046ca: jne 4046b0                          # loop ends
4046cc: vzeroupper
4046cf: ret
```

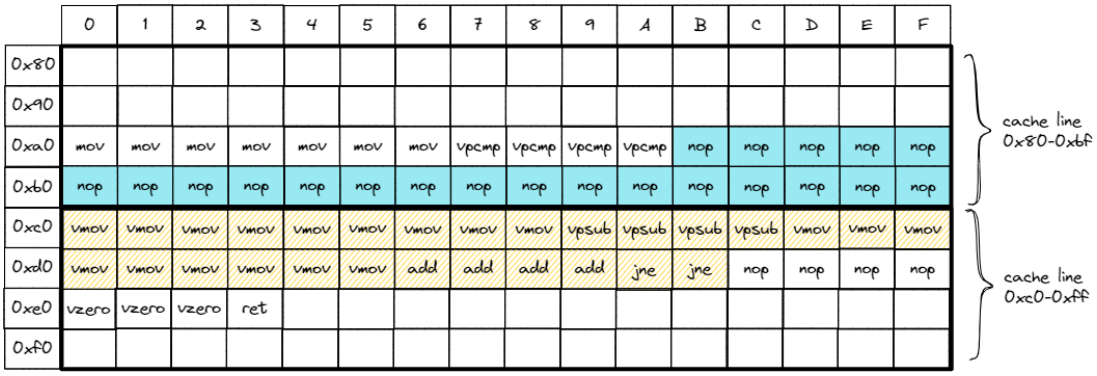
[184] 有关 builtin-expect 的更多信息，请参见此处：https://llvm.org/docs/BranchWeightMetadata.html#builtin-expect。

代码本身相当合理，但布局并不完美（见图 69a ）。对应于循环的指令用黄色斜线突出显示。与数据缓存一样，指令缓存行长度为 64 字节。在图 69 中，粗框表示缓存行边界。请注意，循环跨越多个缓存行：它从缓存行 0x80-0xBF 开始，并在缓存行 0xC0-0xFF 结束。为了获取在循环中执行的指令，处理器需要读取两个缓存行。这些情况通常会导致 CPU 前端的性能问题，尤其是对于上面呈现的小循环。

为了解决这个问题，我们可以使用 NOP 将循环指令向前移动 16 个字节，以便整个循环位于一个缓存行中。图 69b 显示了使用以蓝色突出显示的 NOP 指令执行此操作的效果。有趣的是，即使您在微基准测试中只运行这个热循环，性能影响也是可见的。这有点令人困惑，因为代码量很小，它不应该在任何现代 CPU 上占用 L1I 缓存的大小。图 69b 中布局性能更好的原因解释起来并不简单，并且会涉及大量微体系结构细节，我们在本书中不讨论这些细节。感兴趣的读者可以在 easyperf 博客上的文章 “Code alignment issues: https://easyperf.net/blog/2018/01/18/Code_alignment_issues” 中找到更多信息。[185]



(a) 默认布局



(b) 改进布局

Figure 69: Listing 11.4 中循环的两种不同的代码布局。

默认情况下，LLVM 编译器会识别循环并将它们对齐到 16B 边界，如图 69a 所示。为了达到我们示例的所需代码位置，如图 69b 所示，可以使用 `-mllvm -align-all-blocks=5` 选项，该选项将在目标文件中将每个基本块对齐到 32 字节边界。但是，请谨慎使用此选项，因为它很容易在其他地方降低性能。此选项在执行路径上插入 NOP，这可能会增加程序的开销，尤其是当它们位于关键路径上时。NOP 不需要执行；但是，它们仍然需要从内存中获取、解码和退出。后者还会消耗 FE 数据结构和缓冲区中的空间用于簿记，类似于所有其他指令。LLVM 编译器中还有其他不那么侵入性的选项可用于控制基本块对齐，您可以在 easyperf 博客 post: https://easyperf.net/blog/2018/01/25/Code_alignment_options_in_llvm 中查看这些选项。[186]

LLVM 编译器最近的新增功能是 `[[clang::code_align()]]` 循环属性，它允许开发人员在源代码中指定循环的对齐方式。这提供了对机器代码布局非常细粒度的控制。在引入此属性之前，开发人员不得不求助于一些不太实用的

[185] “Code alignment issues” - https://easyperf.net/blog/2018/01/18/Code_alignment_issues

[186] “Code alignment options in llvm” - https://easyperf.net/blog/2018/01/25/Code_alignment_options_in_llvm

解决方案，例如在源代码中注入内联汇编语句 `asm(".align 64;")`。以下代码展示了如何使用新的 Clang 属性将循环对齐到 64 字节边界：

```
void benchmark_func(int* a) {
  [[clang::code_align(64)]]
  for (int i = 0; i < 32; ++i)
    a[i] += 1;
}
```

尽管 CPU 架构师努力将机器代码布局的影响最小化，但在某些情况下，代码放置（对齐）仍然会影响性能。机器代码布局也是性能测量的主要噪音源之一。它使区分真正的性能改进或回归与意外发生的回归（由代码布局的改变引起）变得更加困难。

11.5 函数拆分

函数拆分的思想是将热点代码与冷代码分开。这种转换通常也被称为函数轮廓化。这种优化对于具有复杂控制流图和热路径中存在大量冷代码块的相对较大函数非常有益。Listing 11.5 中显示了这种转换可能会有益处的代码示例。为了将热路径中的冷基本块移除，我们将它们剪切并粘贴到一个新函数中，并创建一个调用它们的调用。

代码清单：函数拆分：将冷代码轮廓化到新函数。

```
void foo(bool cond1, bool cond2) {
  // 热路径
  if (cond1) {
    /* 大量的冷代码 (1) */
  }
  // 热路径
  if (cond2) {
    /* 大量的冷代码 (2) */
  }
}
```

=>

```
void foo(bool cond1, bool cond2) {
   // 热路径
  if (cond1) {
      cold1();
  }
   // 热路径
  if (cond2) {
      cold2();
  }
}
void cold1() __attribute__((noinline))
  { /* 大量的冷代码 (1) */ }
void cold2() __attribute__((noinline))
  { /* 大量的冷代码 (2) */ }
```

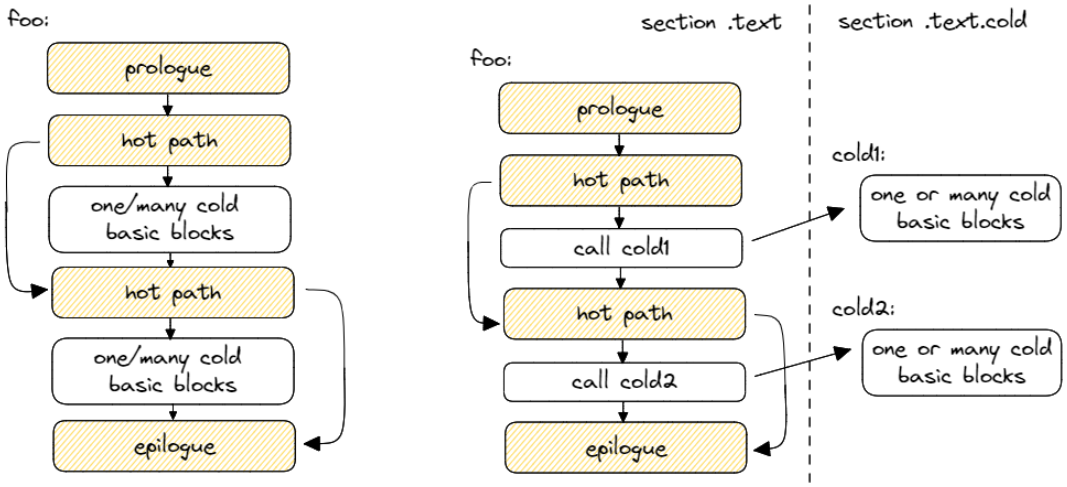
请注意，我们通过使用 `noinline` 属性禁用了冷函数的内联。因为如果没有这个属性，编译器可能会决定内联它，这实际上会撤销我们的转换。或者，我们可以在 `cond1` 和 `cond2` 分支上都应用 `[[unlikely]]` 宏（参见 Section 11.3），以传达给编译器不希望内联 `cold1` 和 `cold2` 函数。

图 70 给出了这种转换的图形表示。因为我们在热路径中只留下了一个 `CALL` 指令，所以下一个热指令很可能会与上一个指令驻留在同一个缓存行中。这提高了 CPU 前端数据结构（如 I-cache 和 DSB）的利用率。

轮廓化的函数应该创建在 `.text` 段之外，例如在 `.text.cold` 中。如果函数从不被调用，这样做可以提高内存占用，因为它在运行时不会加载到内存中。

11.6 函数重排序

根据前面各节描述的原则，可以将热点函数分组，进一步提高 CPU 前端缓存的利用率。当热点函数被分组在一起时，它们开始共享缓存行，这减少了代码占用空间，即 CPU 需要获取的缓存行总数。



(a) 默认布局　　(b) 改进的布局

Figure 70: 将冷代码拆分到单独的函数中。

图 71 给出了重新排列热点函数 `foo`、`bar` 和 `zoo` 的图形表示。图像上的箭头显示了最频繁的调用模式，即 `foo` 调用 `zoo`，然后 `zoo` 调用 `bar`。在默认布局中（见图 71a ），热点函数不相邻，它们之间有一些冷函数。因此，两个函数调用的序列（`foo -> zoo -> bar`）需要读取四个缓存行。

我们可以重新排列函数的顺序，使得热点函数彼此靠近（见图 71b ）。在改进的版本中，`foo`、`bar` 和 `zoo` 函数的代码适合于三个缓存行。另外，注意函数 `zoo` 现在根据函数调用的顺序被放置在 `foo` 和 `bar` 之间。当我们从 `foo` 调用 `zoo` 时，`zoo` 的开始已经在 I-cache 中。

与之前的优化类似，函数重排序提高了 I-cache 和 DSB-cache 的利用率。当存在许多小型热点函数时，这种优化效果最佳。

链接器负责将程序的所有函数布局在生成的二进制输出中。虽然开发人员可以尝试自己重新排列程序中的函数，但不能保证所需的物理布局。几十年来，人们一直使用链接器脚本来实现这个目标。如果你使用的是 GNU 链接器，这仍然是一种行之有效的方法。Gold 链接器 (`ld.gold`) 对这个问题有了更简单的解决方案。要在 Gold 链接器中获得二进制文件中函数的所需顺序，可以首先使用 `-ffunction-sections` 标志编译代码，这会将每个函数放入一个单独的节中。然后使用 `--section-ordering-file=order.txt` 选项提供一个带有按所需最终布局排序的函数名称列表的文件。LLD 链接器也具有相同的特性，它是 LLVM 编译器基础设施的一部分，并通过 `--symbol-ordering-file` 选项访问。

解决将热点函数分组在一起的问题的一个有趣方法是由 Meta 的工程师在 2017 年引入的。他们实现了一个名为 HFSort 的工具[187]，它根据分析数据自动生成节顺序文件 [Ottoni & Maher, 2017]。使用这个工具，他们观察到大型分布式云应用程序，如 Facebook、百度和维基百科的性能提高了 2%。HFSort 已经集成到了 Meta 的 HHVM、LLVM BOLT 和 LLD 链接器中[188]。从那时起，该算法首先被 HFSort+ 取代，最近又被 Cache-Directed Sort (CDSort[189]) 所取代，对于具有大型代码占用空间的工作负载带来了更多改进。

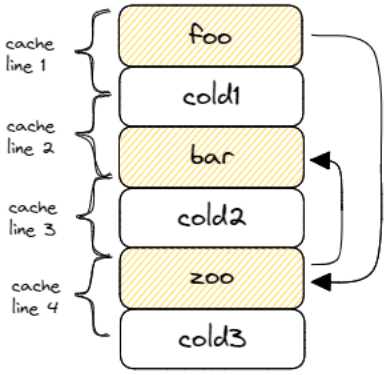
11.7 基于性能分析引导的优化（Profile Guided Optimizations）

编译程序和生成最佳汇编代码都是关于启发式的。代码转换算法有很多特殊情况，旨在在特定情况下实现最佳性能。对于编译器做出的许多决策，它会尝试根据一些典型案例猜测最佳选择。例如，当决定是否内联特定函数时，编译

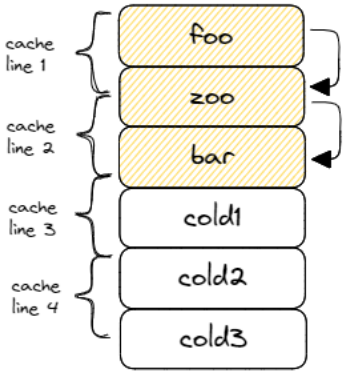
[187] HFSort - https://github.com/facebook/hhvm/tree/master/hphp/tools/hfsort

[188] LLD 中的 HFSort - https://github.com/llvm-project/lld/blob/master/ELF/CallGraphSort.cpp

[189] LLVM 中的 Cache-Directed Sort - https://github.com/llvm/llvm-project/blob/main/llvm/lib/Transforms/Utils/CodeLayout.cpp



(a) 默认布局



(b) 改进的布局

Figure 71: 重新排列热点函数。

器可能会考虑该函数被调用的次数。问题是编译器事先并不知道这一点。它首先需要运行程序才能找出答案。如果没有运行时信息，编译器就必须猜测。

这就是分析 (profiling) 信息派上用场的时候。有了分析 (profiling) 信息，编译器可以做出更好的优化决策。大多数编译器中有一组转换，可以根据反馈给他们的分析 (profiling) 数据调整算法。这组转换称为分析 (profiling) 优化 (PGO)。当分析 (profiling) 数据可用时，编译器可以用它来指导优化。否则，它将退回到使用其标准算法和启发式。有时在文献中，您可以找到术语反馈定向优化 (FDO)，它指的是与 PGO 相同的东西。

图 72 显示了使用 PGO 的传统工作流程，也称为 插桩化的 (instrumented) PGO。首先，您编译您的程序并告诉编译器自动检测代码。这将在函数中插入一些记账代码以收集运行时统计信息。第二个步骤是使用代表应用程序典型工作负载的输入数据运行插桩化的 (instrumented) 二进制文件。这将生成分析 (profiling) 数据，一个包含运行时统计信息的新文件。它是一个原始转储文件，其中包含有关函数调用计数、循环迭代计数和其他基本块命中计数的信息。此工作流程的最后一步是使用分析 (profiling) 数据重新编译程序以生成优化的可执行文件。

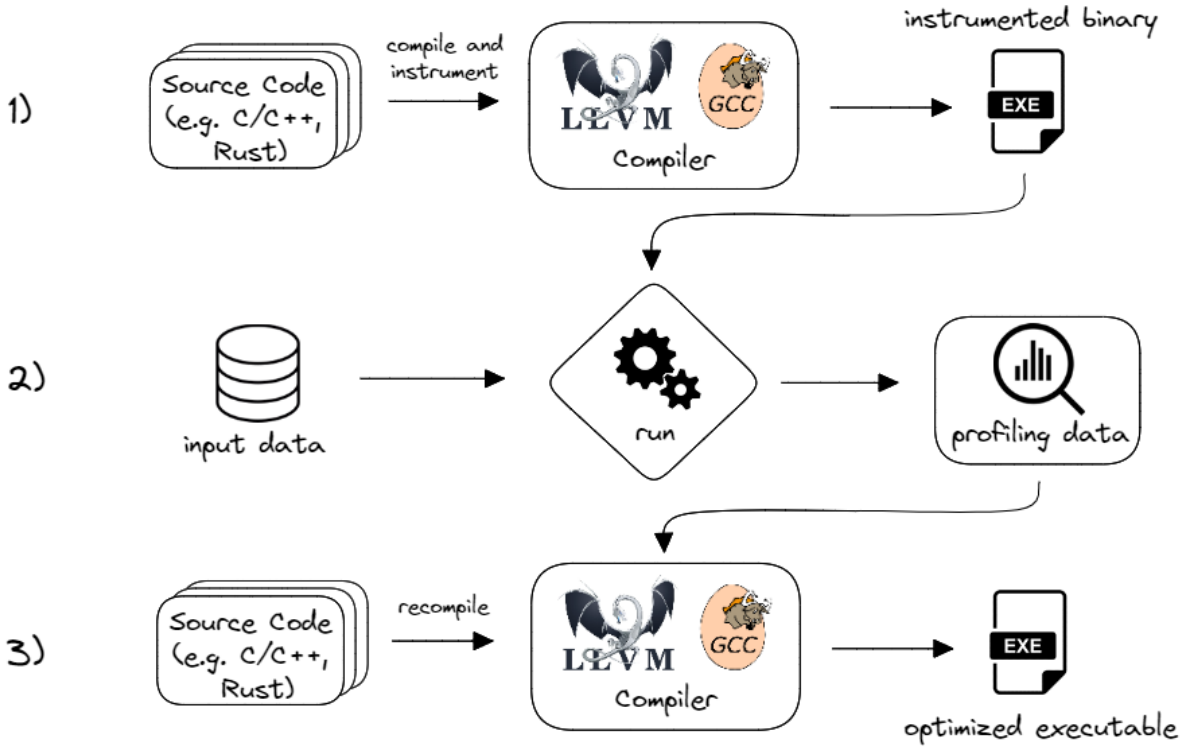


Figure 72: 使用工具 PGO 的工作流程.

开发人员可以通过使用 `-fprofile-instr-generate` 选项构建程序来启用 LLVM 编译器中的 PGO 检测（步骤 1）。这将指示编译器检测代码，从而在运行时收集分析 (profiling) 信息。之后，LLVM 编译器可以使用 `-fprofile-instr-use` 选项使用分析 (profiling) 数据重新编译程序并输出经过 PGO 调整的二进制文件。clang 中使用 PGO 的指南在文档: https://clang.llvm.org/docs/UsersManual.html#profiling-with-instrumentation 中进行了描述。[190] GCC 编译器使用不同的选项集：`-fprofile-generate` 和 `-fprofile-use`，如 文档: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html#Optimize-Options 所述。[191]

PGO 帮助编译器改进函数内联、代码放置、寄存器分配和其他代码转换。PGO 主要用于具有大型代码库的项目，例如 Linux 内核、编译器、数据库、网络浏览器、视频游戏、生产力工具等。对于拥有数百万行代码的应用程序，它是改善机器代码布局的唯一实践方法。使用分析 (profiling) 优化使生产工作负载的性能提高 10-25% 并不少见。

[190] Clang 中的 PGO - https://clang.llvm.org/docs/UsersManual.html#profiling-with-instrumentation

[191] GCC 中的 PGO - https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html#Optimize-Options

虽然许多软件项目将插桩化的 (instrumented) PGO 作为其构建过程的一部分，但采用率仍然很低。这有几个原因。主要原因是插桩化的 (instrumented) 可执行文件的巨大运行时开销。运行插桩化的 (instrumented) 二进制文件和收集分析 (profiling) 数据经常会降低 5-10 倍的速度，这使构建步骤更长，并且阻止直接从生产系统（无论是在客户端设备还是云端）收集优化分析文件。不幸的是，您无法一次收集分析 (profiling) 数据并将其用于所有未来的构建。随着应用程序源代码的演变，优化分析文件数据会变得陈旧（不同步），需要重新收集。

PGO 流程的另一个注意事项是，编译器应该只使用应用程序将如何使用的代表性场景进行训练。否则，您最终可能会降低程序的性能。编译器会“盲目地”使用您提供的优化分析文件数据。它假设程序无论输入数据是什么都始终表现相同。PGO 的用户应该谨慎选择用于收集优化分析文件数据（步骤 2）的输入数据，因为在改进应用程序的一个用例的同时，其他用例可能会被悲观化。幸运的是，它不必仅仅是单个工作负载，因为来自不同工作负载的优化分析文件数据可以合并在一起，以代表应用程序的一组用例。

谷歌在 2016 年率先提出了另一种基于样本的 PGO 解决方案。[Chen et al., 2016] 除了检测代码之外，还可以从标准优化分析文件工具（例如 Linux `perf`）的输出中获取优化分析文件数据。谷歌开发了一个名为 AutoFDO: https://github.com/google/autofdo[192] 的开源工具，可以将 Linux `perf` 生成的采样数据转换为 GCC 和 LLVM 等编译器可以理解的格式。

与带插桩的 PGO 相比，这种方法有一些优点。首先，它消除了 PGO 构建工作流程的一个步骤，即步骤 1，因为无需构建带有插桩的二进制文件。其次，优化分析文件数据收集运行在已经优化的二进制文件上，因此运行时开销要低得多。这使得可以在生产环境中更长时间地收集优化分析文件数据。由于这种方法基于硬件收集，它还支持使用带插桩的 PGO 无法实现的新型优化。一个例子是分支到 cmov 转换，这是一个用条件移动替换条件跳转以避免分支预测错误开销的转换 (参见 Section 10.3)。为了有效地执行此转换，编译器需要知道原始分支的错误预测频率。此信息可在现代 CPU（Intel Skylake+）上的基于样本的 PGO 中获得。

下一个创新想法来自 Meta，它在 2018 年年中开源了其名为 BOLT: https://code.fb.com/data-infrastructure/accelerate-large-scale-applications-with-bolt/ 的二进制优化工具。[193] BOLT 在已经编译的二进制文件上工作。它首先反汇编代码，然后使用 Linux perf 等采样分析器收集的优化分析文件信息进行各种布局转换，然后重新链接二进制文件。[Panchenko et al., 2018] 截至今天，BOLT 拥有超过 15 个优化通道，包括基本块重新排序、函数拆分和重新排序等。与传统 PGO 类似，BOLT 优化的主要候选是遭受许多指令缓存和 iTLB 未命中折磨的程序。自 2022 年 1 月起，BOLT 成为 LLVM 项目的一部分，并作为独立工具提供。

谷歌在 BOLT 引入几年后开源了其名为 Propeller: https://github.com/google/llvm-propeller 的二进制重新链接工具。它具有类似的目的，但它不反汇编原始二进制文件，而是依赖于链接器输入，因此可以分布在多个机器上以实现更好的扩展和更少的内存消耗。BOLT 和 Propeller 等后链接优化器可以与传统 PGO (和 LTO) 结合使用，通常可以提供额外 5-10% 的性能提升。此类技术开辟了基于硬件遥测的新型二进制重写优化。

11.8 减少 ITLB 未命中

调整前端效率的另一个重要领域是内存地址的虚拟到物理地址转换。这些转换主要由 TLB（参见 Section 3.7.1）提供服务，TLB 在专用条目中缓存最近使用的内存页转换。当 TLB 无法处理翻译请求时，将进行耗时的内核页表页面遍历，为每个引用的虚拟地址计算正确的物理地址。每当您在 TMA 摘要中看到高比例的 ITLB 开销时，本节中的建议可能派上用场。

通常情况下，相对较小的应用程序不易受到 ITLB 未命中的影响。例如，Golden Cove 微架构可以在其 ITLB 中覆盖高达 1MB 的内存空间。如果您的应用程序的机器代码适合 1MB，您应该不会受到 ITLB 未命中的影响。当应用程序中频繁执行的部分分散在内存周围时，问题就开始出现。当许多函数开始频繁相互调用时，它们会开始争夺 ITLB 中的条目。一个例子是 Clang 编译器，在撰写本文时，它的代码部分大约为 60MB。在运行主流 Intel CoffeeLake 处

[192] AutoFDO - https://github.com/google/autofdo

[193] BOLT - https://code.fb.com/data-infrastructure/accelerate-large-scale-applications-with-bolt/

理器的笔记本电脑上，ITLB 开销约为 7%，这意味着 7% 的周期被浪费在处理 ITLB 未命中上：执行要求苛刻的页面遍历和填充 TLB 条目。

另一组经常受益于使用大页的大内存应用程序包括关系数据库（例如 MySQL、PostgreSQL、Oracle）、托管运行时（例如 Javascript V8、Java JVM）、云服务（例如网络搜索）、网络工具（例如 node.js）。将代码段映射到大页可以将 ITLB 未命中数量减少高达 50% [Suresh Srinivas, 2019]，从而为某些应用程序带来高达 10% 的加速。但是，与许多其他功能一样，大页并不适用于所有应用程序。可执行文件只有几 KB 大的小程序最好使用常规 4KB 页面而不是 2MB 大页；这样，内存可以更有效地利用。

减少 ITLB 压力的总体思路是将应用程序性能关键代码的部分映射到 2MB（大页）上。但通常情况下，整个应用程序的代码部分会重新映射以简化操作，或者如果您不知道哪些函数是热门函数。要进行这种转换，关键要求是代码部分与 2MB 边界对齐。在 Linux 上，这可以通过两种不同的方式实现：使用附加链接器选项重新链接二进制文件或在运行时重新映射代码部分。这两个选项都展示在 easyperf.net 博客[194] 上。据我们所知，它在 Windows 上是不可能的，因此我们只展示如何在 Linux 上进行操作。

第一个选项可以通过使用 `-Wl,-zcommon-page-size=2097152 -Wl,-zmax-page-size=2097152` 选项链接二进制文件来实现。这些选项指示链接器将代码段放在 2MB 边界，以便启动时由加载器将其放置在 2MB 页面上。这种放置的缺点是链接器将被迫插入多达 2MB 的填充（浪费）字节，使二进制文件更加臃肿。以 Clang 编译器为例，它使二进制文件的大小从 111MB 增加到 114MB。重新链接二进制文件后，我们在 ELF 二进制头中设置一个特殊位，该位确定文本段是否应默认由大页支持。最简单的方法是使用 libhugetlbfs: https://github.com/libhugetlbfs/libhugetlbfs/blob/master/HOWTO[195] 软件包中的 `hugeedit` 或 `hugectl` 工具。例如：

```
# 永久设置 ELF 二进制头中的特殊位。
$ hugeedit --text /path/to/clang++
# 代码段将默认使用大页加载。
$ /path/to/clang++ a.cpp

# 在运行时覆盖默认行为。
$ hugectl --text /path/to/clang++ a.cpp
```

第二个选项是在运行时重新映射代码段。此选项不需要代码段与 2MB 边界对齐，因此可以在不重新编译应用程序的情况下工作。这种方法背后的 idea 是在程序启动时分配大页并将所有代码段转移到那里。该方法的参考实现是在 iodlr: https://github.com/intel/iodlr[196] 中实现的。一个选项是从您的 `main` 函数调用该功能。另一个更简单的方法是构建动态库并在命令行中预加载它：

```
$ LD_PRELOAD=/usr/lib64/liblppreload.so clang++ a.cpp
```

虽然第一种方法只能使用显式大页，但使用 `iodlr` 的第二种方法既适用于显式大页，也适用于透明大页。有关如何在 Windows 和 Linux 上启用大页的说明，请参见附录 C。

除了采用大页之外，还可以使用优化 I-cache 性能的标准技术来改善 ITLB 性能。例如，重新排序函数以更好地定位热门函数，通过链接时优化 (LTO/IPO) 减少热门区域的大小，使用概要引导优化 (PGO) 和 BOLT，以及减少激进内联。

BOLT 提供了 `-hugify` 选项，可以根据优化分析文件数据自动将大页用于热门代码。使用此选项时，`llvm-bolt` 将注入代码，在运行时将热门代码放在 2MB 页面上。该实现利用了 Linux 透明大页 (THP)。这种方法的优点是只有少部分代码映射到大页，所需的大页数量最小化，因此页面碎片减少。

[194] “使用大页为代码带来的性能优势” - https://easyperf.net/blog/2022/09/01/Utilizing-Huge-Pages-For-Code.

[195] libhugetlbfs - https://github.com/libhugetlbfs/libhugetlbfs/blob/master/HOWTO.

[196] iodlr 库 - https://github.com/intel/iodlr.

11.9 案例研究：测量代码足迹 (Measuring Code Footprint)

正如我们在本章中多次提到的，代码布局优化对具有大量代码的应用程序影响最大。阐明程序中热代码大小不确定的最佳方法是测量其 代码足迹，我们将其定义为程序执行期间触及的机器指令的字节/缓存行/页面数量。

大型代码足迹本身并不一定会对性能产生负面影响。代码足迹不是决定性的指标，它不会立即告诉您是否存在问题。尽管如此，它已被证明作为性能分析中一个有用的额外数据点。结合 TMA 的`Frontend_Bound`, `L1-instruction cache miss rate`(L1 指令缓存未命中率)和其他指标，它可能会加强投资时间优化应用程序机器代码布局的论据。

目前，可以可靠地测量代码足迹的工具还很少。在本案例研究中，我们将展示 perf-tools: https://github.com/aayasin/perf-tools，[197] 一个基于 Linux `perf` 构建的开源性能分析工具集。为了估计[198] 代码足迹，`perf-tools` 利用了英特尔的 LBR（参见 Section 6.2），因此目前它不能在 AMD 或 ARM 架构的系统上工作。下面是一个收集代码足迹数据的示例命令：

```
$ perf-tools/do.py profile --profile-mask 100 -a <your benchmark>
```

其中，`--profile-mask 100` 启动 LBR 采样，`-a` 允许您指定要运行的程序。此命令将收集代码足迹以及其他各种数据。我们不显示工具的输出，好奇的读者可以自行尝试。

我们选取了一组四个基准测试：Clang C++ 编译、Blender 光线追踪、Cloverleaf 流体动力学和 Stockfish 国际象棋引擎；这些工作负载您应该已经从 Section 4.11 分析其性能特征的地方熟悉了。我们在基于英特尔 Alderlake 的处理器上使用与 Section 4.11 中相同的命令运行它们。正如预期的那样，在基于 Skylake 的机器上运行相同基准测试获得的代码足迹数字与 Alderlake 运行的结果非常相似。代码足迹取决于程序和输入数据，而不是特定机器的特性，因此结果在不同架构上应该看起来相似。

四个基准测试的结果分别列在表 [8] 中。二进制和 `.text` 大小是使用标准的 Linux `readelf` 工具获得的，而其他指标是使用 `perf-tools` 收集的。如果一个指令内存位置被命中至少一次，该工具将其视为“非冷”，因此，`non-cold code footprint [KB]`是程序触及的包含机器指令的千字节数。指标`non-cold code 4KB-pages`告诉我们程序触及的包含机器指令的非冷 4KB 页数。它们一起帮助我们理解这些非冷内存位置的密度或稀疏性。一旦我们深入研究这些数字，这一点就会变得清晰。最后，我们还提供了前端瓶颈百分比，这是一个您应该已经从 Section 6.1 中了解的关于 TMA 的指标。

Table 8: 案例研究中使用的基准测试的代码足迹。

Metric	Clang17 编译	Blender	CloverLeaf	Stockfish
Binary size [KB]	113844	223914	672	39583
`.text` size [KB]	67309	133009	598	238
non-cold code footprint [KB]	5042	313	104	99
non-cold code 4KB-pages	6614	546	104	61
Frontend Bound, Alderlake-P [%]	52.3	29.4	5.3	25.8

解释：

- **二进制大小 (Binary size):** 指的是应用程序的可执行文件大小。
- **`.text` 大小:** 指的是可执行文件中代码段的大小。

[197] perf-tools - https://github.com/aayasin/perf-tools

[198] `perf-tools`收集的代码足迹数据是不精确的，因为它是基于 LBR 记录的采样。不幸的是，其他工具，如英特尔的`sde -footprint`，不提供代码占用。然而，自己编写一个基于 pin 的工具来测量准确的代码占用并不难。

- **非冷代码足迹 (non-cold code footprint):** 指的是程序执行过程中触及的代码部分的大小，以千字节为单位。
- **非冷代码 4KB 页 (non-cold code 4KB-pages):** 指的是非冷代码占用的 4KB 页数。
- **前端瓶颈 (Frontend Bound) Alderlake-P:** 指的是由于等待指令而导致的性能瓶颈，以百分比表示。

观察：

- Clang17 编译的非冷代码足迹最大，其次是 Stockfish 和 CloverLeaf。
- Blender 虽然 `.text` 大小很大，但其非冷代码足迹相对较小。
- CloverLeaf 的非冷代码 4KB 页利用率最高，其次是 Stockfish 和 Clang17。
- Clang17 的前端瓶颈问题最为严重，其次是 Stockfish 和 Blender。

分析：

- 代码足迹可以帮助我们了解应用程序对 CPU 前端的压力。
- 非冷代码 4KB 页利用率可以反映代码布局的紧凑程度。
- 前端瓶颈指标可以帮助我们识别性能瓶颈。

注意：

- 此分析不考虑应用程序外部的代码，例如动态链接库。

让我们先来看看二进制和`.text`的大小。与 Clang17 和 Blender 相比，CloverLeaf 是一个非常小的应用程序；Stockfish 嵌入的神经网络文件占了大部分二进制文件，但其代码部分相对较小；Clang17 和 Blender 拥有庞大的代码库。`.text size` 指标是我们的应用程序的上限，即我们假设[199] 代码足迹不应超过 `.text` 大小。

通过分析代码足迹数据，我们可以做出一些有趣的观察。首先，尽管 Blender 的 `.text` 部分非常大，但不到 1% 的 Blender 代码是非冷的：133 MB 中只有 313 KB。因此，仅仅因为二进制文件很大，并不意味着应用程序会遭受 CPU 前端瓶颈。真正重要的是热代码的数量。对于其他基准测试，这个比率更高：Clang17 7.5%，CloverLeaf 17.4%，Stockfish 41.6%。从绝对数字来看，Clang17 编译触及的机器指令字节数比其他三个应用程序高出一个数量级。

其次，让我们检查表中的“非冷代码 4KB 页”行。对于 Clang17，5042 KB 的非冷代码分布在 6614 个 4KB 页面上，这给我们提供了 `5042 / (6614 * 4) = 19%` 的页面利用率。这个指标告诉我们代码的热点部分的密度/稀疏性。每个热点缓存行越靠近另一个热点缓存行，就需要更少的页面来存储热点代码。页面利用率越高越好。本章前面讨论的基本块放置和函数重新排序是提高页面利用率的转换的完美例子。对于其他基准测试，比率为：Blender 14%，CloverLeaf 25%，Stockfish 41%。

现在我们已经量化了四个应用程序的代码足迹，可能会考虑 L1 指令缓存和 L2 缓存的大小以及热代码是否适合。在我们的 Alderlake 处理器上，L1-I cache 只有 32 KB，不足以完全覆盖我们分析过的任何基准测试。但请记住，在本节开头我们说过，大型代码足迹并不直接指向问题。是的，大型代码库会给 CPU 前端带来更大的压力，但指令访问模式也对性能至关重要。与数据访问相同的局部性原则也适用。这就是为什么我们将其与 Topdown 分析中的前端瓶颈指标结合在一起。

对于 Clang17，5 MB 的非冷代码导致了一个巨大的 52.3% 的前端瓶颈性能瓶颈：超过一半的周期都在等待指令。在所有呈现的基准测试中，它从 PGO 类型优化中获益最多。CloverLeaf 不存在指令获取效率低下的问题；它 75% 的分支都是向后跳转，这表明它们可能是反复执行的相对较小的循环。Stockfish 虽然与 CloverLeaf 具有大致相同的非冷代码足迹，但却对 CPU 前端提出了更大的挑战 (25.8%)。它有更多的间接跳转和函数调用。最后，Blender 比 Stockfish 具有更多的间接跳转和调用。由于进一步的调查超出了本案例研究的范围，因此我们在此停止分析。对于有兴趣继续分析的读者，我们建议根据 TMA 方法深入研究前端瓶颈类别，并查看诸如 `ICache_Misses`, `ITLB_Misses`, `DSB coverage` 等指标。

[199] 这并不总是正确的: 一个应用程序本身可能很小，但需要调用多个其他动态链接的库，或者它可能大量使用内核代码。

另一个研究代码足迹的有用工具是 llvm-bolt-heatmap: https://github.com/llvm/llvm-project/blob/main/bolt/docs/Heatmaps.md[200]，它是 llvm 的 BOLT 项目的一部分。该工具可以生成代码热图，让您细粒度地了解应用程序的代码布局。其主要用途是 1) 评估原始代码布局是否紧凑以及是否可以优化，2) 确保优化后的代码布局在实际负载下仍然紧凑。

问题与练习

1. 解决 `perf-ninja::pgo` 实验作业。
2. 尝试为代码段使用大页。选择一个大型应用程序（如果能够访问源代码则更佳，但不是必需的），其二进制文件大小超过 100MB。尝试使用 Section 11.8 中描述的一种方法将其代码段重新映射到大页上。观察性能的任何变化，`/proc/meminfo` 中的大页分配情况，以及测量 ITLB 命中和未命中的 CPU 性能计数器。
3. 假设你有一段代码，其中包含一个循环中的 C++ switch 语句。你已经对代码进行了分析，并发现 switch 语句中的一个特定情况占了 70% 的时间。其他 40 种情况每种使用时间不到 3%，另外 20 种情况从未发生。你会如何优化这个 switch/循环的性能？
4. 在你日常使用的应用程序上运行 PGO、llvm-bolt 或 Propeller。比较“优化前”和“优化后”的性能分析，以了解加速来自何处。

AI 回答:

1. **解决 `perf-ninja::pgo` 实验作业**：
   - 首先，确保你有一个支持 PGO（Profile Guided Optimizations）的编译环境，比如 GCC 或 Clang。
   - 编译你的程序，启用 PGO 插桩（instrumentation），这通常涉及到添加特定的编译器标志，如`-fprofile-instr-generate`。
   - 使用你的程序运行一系列代表性的工作负载，以收集性能数据。这通常涉及到运行程序多次，或者使用特定的工具来收集运行时数据。
   - 分析收集到的性能数据，这可能包括函数调用次数、循环迭代次数等。
   - 使用这些数据重新编译你的程序，启用 PGO 使用（`-fprofile-use`）来生成优化后的二进制文件。
   - 测试优化后的程序，比较性能提升。
2. **尝试为代码段使用大页**：
   - 选择一个大型应用程序，确保其二进制文件大小超过 100MB。
   - 在 Linux 系统上，你可以使用`madvise`系统调用来建议内核使用大页。例如，`madvise((void *)address, size, MADV_HUGEPAGE)`。
   - 另一种方法是在编译时使用特定的链接器选项，如`-Wl,-zmax-page-size=2M`，这会告诉链接器尝试将代码段放置在 2MB 大页上。
   - 观察性能变化，可以通过运行基准测试来完成。
   - 查看`/proc/meminfo`文件来确认大页的使用情况。
   - 使用性能分析工具（如`perf`）来测量 ITLB 的命中和未命中情况。
3. **优化 C++ switch 语句**：
   - 对于频繁执行的特定情况，可以考虑将其提取到一个单独的函数中，以减少 switch 语句的复杂性。
   - 对于从未发生的分支，可以完全移除它们，以简化 switch 语句。
   - 对于使用频率较低的分支，可以考虑使用跳转表（jump table）或者基于范围的 switch 来优化。
   - 如果 switch 语句基于某些可预测的模式（如枚举类型），可以考虑使用位掩码或查找表来加速决策过程。
4. **运行 PGO、llvm-bolt 或 Propeller**：
   - 对于 PGO，按照上述步骤 1 的指导进行。
   - 对于 llvm-bolt，可以使用`llvm-bolt`工具对二进制文件进行优化。这通常涉及到反汇编代码，应用优化，然后重新链接。

[200] llvm-bolt-heatmap - https://github.com/llvm/llvm-project/blob/main/bolt/docs/Heatmaps.md

- 对于 Propeller，这是一个二进制重链接工具，它依赖于链接器输入，可以在多个机器上分布式运行。
- 在优化前后，使用性能分析工具（如`perf`）来比较性能指标，如 CPU 周期、缓存命中率等，以了解性能提升的来源。

章节总结

CPU 前端优化总结见表 9。

Table 9: CPU 前端优化总结。

转换	如何转换?	为什么有用?	最适合于	由谁完成
基本块放置 (Basic block placement)	维护热点 代码	未取分支 更便宜; 更好的缓 存利用率	任何代码，尤其是具有大量分支 的代码	编译器
基本块对齐 (Basic block alignment)	使用 NOPS 对齐移动 热点代码	更好的缓存 利用率	热循环	编译器
函数拆分 (Function splitting)	将冷代码块拆 分并放置在单 独的函数中	更好的缓存 利用率	具有复杂 CFG 的函数，当热 部分之间有 大块冷代码时	编译器
函数重新排序 (Function splitting)	将热点函数 分组在一起	更好的缓存 利用率	许多小型热点 函数	链接器

- 代码布局改进经常被低估，最终被忽略和遗忘。CPU 前端性能问题，例如 I-cache 和 ITLB 未命中，浪费了大量周期，尤其是对于具有大型代码库的应用程序。但即使是小型和中型应用程序也可以从优化机器代码布局中受益。
- 当开发人员试图改善应用程序性能时，他们通常不会首先关注它。他们更喜欢从低垂的果实开始，例如循环展开和向量化。然而，要知道仅仅通过更好的机器代码布局，您就可能获得额外的 5-10%，这仍然很有用。
- 如果您可以为您的应用程序制定一套典型用例，则通常最好使用 LTO、PGO、BOLT 和其他工具。对于大型应用程序，它是改善机器代码布局的唯一实用方法。